# TECHNICAL DESIGN DOCUMENT (TDD)

Jewellery Gold Accountability, Reconciliation & Anomaly Detection Platform

| | |
|---|---|
| **Target Domain** | Coimbatore Jewellery Enterprise (5 Showrooms, 1 Central Workshop, ~40 Karigars) |
| **Tech Stack** | Spring Boot 3.3 (Java 21) + React 18 (TypeScript) + FastAPI (Python 3.13) + PostgreSQL 16+ |
| **Author / Candidate** | Ayush — Isocode Labs Software Engineering Internship Assignment |
| **GitHub Repository** | https://github.com/Ayush5424/jewellery-gold-accountability |
| **Version & Date** | Version 1.0.0 Production-Ready \| November 2026 |

## 1. Executive Summary & Coimbatore Business Requirements

A premier jewellery manufacturing and retail enterprise headquartered in Coimbatore, Tamil Nadu, operates 5 distinct retail showrooms, 1 centralized manufacturing workshop, and coordinates with approximately 40 skilled artisan karigars. Prior to this platform, gold accounting was performed through manual job cards, paper scale slips, and fragmented spreadsheets. This led to severe operational vulnerabilities including unexplained fine-gold variances, delayed reconciliation weeks after job delivery, lack of benchmark drift detection, and inability to account for every gram of gold across the business in real time.

The Jewellery Gold Accountability System was engineered to provide complete, authoritative, and auditable accountability for every single gram of gold to three decimal places (0.001 g). The core business mandate answers with certainty: How much gold was issued? How much has each karigar returned? How much remains outstanding? How much was lost during manufacture? Did loss exceed contractual wastage allowances? And what is the authoritative live stock across the entire business?

## 2. System Architecture & Tech Stack

The platform employs an enterprise three-tier distributed architecture ensuring strong separation of concerns, strict authorization, and zero client-side calculation authority:

| Layer | Technology | Architectural Role & Guarantees |
|---|---|---|
| **Frontend SPA** | React 18, TypeScript 5, Vite, Tailwind CSS | Responsive UI for desktop/mobile, state management, role-based screen rendering, interactive reconciliation workflows, and Tally XML export. |
| **Backend Core** | Spring Boot 3.3.4 (Java 21), Spring Data JPA | Authoritative accounting engine, immutable transaction ledger, lifecycle state machines, role security, and RESTful service APIs. |
| **Database** | PostgreSQL 16+ / 18, Hibernate | Relational persistence, foreign key enforcement, pessimistic locking on jobs, and complete audit logging. |
| **AI Vision Service** | Python 3.13, FastAPI, OpenCV, Pillow, Pydantic | Draft extraction of digital scale slips via regex pattern recognition, confidence scoring, missing-field detection, and non-blocking background OCR. |
| **ERP Integration** | Native Tally XML Stock Journal Format | Direct integration with Tally ERP 9 / Tally Prime for accounting stock transfer journals without third-party plugins. |

## 3. Authoritative Backend Ledger & Data Model

The single source of truth is the immutable, append-only **gold_transactions** ledger. Historical transactions can never be modified or deleted. Any administrative adjustment requires an explicit **ADJUSTMENT_REVERSAL** transaction with mandatory audit remarks.

**Entity Model Highlights:**
• **User**: System credentials, BCrypt hashes, and assigned roles (ROLE_FINANCE_HEAD, ROLE_ISSUE_CLERK, ROLE_KARIGAR).
• **Karigar**: Artisan master record with unique code (KRG-xxx), phone, address, status, and linked user identity.
• **Job**: Manufacturing work order containing job code (JOB-xxx), target item type, issue date, status, allowed wastage %, and reconciled metrics.
• **GoldTransaction**: Ledger entry tracking transaction type, gross weight, purity %, calculated fine gold, tare weight, scale slip image URL, and clerk user ID.
• **LossAlert**: System-generated exception raised when reconciled loss exceeds contractual allowance or historical drift exceeds threshold.

### Job Lifecycle State Machine

Jobs advance through a deterministic state machine: **OPEN** (work order created and raw gold issued) → **IN_PROGRESS** (partial receipts of jewellery, scrap, or filings logged) → **READY_FOR_RECONCILIATION** (artisan declares production complete; all returned materials weighed) → **RECONCILED** (clerk/finance head finalizes job; loss formulas execute; unrecoverable loss finalized; alerts triggered if breach occurs).

## 4. Strict Accounting Formulas & Sample Calculation Verification

All calculations strictly utilize java.math.BigDecimal with RoundingMode.HALF_UP to completely prevent binary floating-point representation drift. Gross weights are recorded to 3 decimal places (0.001 g) and purities to 2 decimal places (0.01 %).

**Core Mathematical Formulas:**
1. Fine Gold Weight (g) = round(Gross Weight * (Purity % / 100), 3)
2. Allowed Loss (g) = round(Issued Fine Gold * (Wastage Allowance % / 100), 3)
3. Outstanding Balance (g) = Issued Fine Gold - Total Recovered Fine Gold
4. Reconciled Actual Loss (g) = Issued Fine Gold - Total Recovered Fine Gold
5. Excess Loss (g) = max(0.000, Actual Loss - Allowed Loss)

### Assignment Specification Sample Calculation Verification

The exact sample calculation mandated in the assignment specification was verified in automated unit test GoldAccountingCalculatorTest.testPart3AssignmentSampleCalculation:

| Stage | Input Parameters | Exact Mathematical Evaluation | Verified Ledger Output |
|---|---|---|---|
| **Issue Transaction** | Gross: 120.000 g<br>Purity: 91.60% (22K)<br>Wastage Allow: 3.00% | Fine = 120.000 * 0.9160 = 109.920 g<br>Allowed Loss = 109.920 * 0.03 = 3.2976 → 3.298 g | **Issued Fine: 109.920 g**<br>Allowed Loss: 3.298 g |
| **Receipt 1 (Jewellery)** | Gross: 112.450 g<br>Purity: 91.60% | Fine = 112.450 * 0.9160 = 103.0042 → 103.004 g | **Recovered: 103.004 g** |
| **Receipt 2 (Scrap)** | Gross: 3.300 g<br>Purity: 91.60% | Fine = 3.300 * 0.9160 = 3.0228 → 3.023 g | **Recovered: 3.023 g** |
| **Total Recovered** | Jewellery + Scrap | 103.004 + 3.023 = 106.027 g | **Recovered: 106.027 g** |
| **Pre-Reconciliation** | Issued - Recovered | 109.920 - 106.027 = 3.893 g | **Outstanding: 3.893 g** |
| **Final Reconciliation** | Actual Loss vs Allowed | Loss = 3.893 g; Allowed = 3.298 g<br>Excess = 3.893 - 3.298 = 0.595 g | **Actual Loss: 3.893 g**<br>**Excess Loss: 0.595 g (BREACH)** |

## 5. Dual-Engine Loss & Drift Detection

To protect against both acute wastage spikes and gradual systemic theft, the platform deploys two complementary engines:
1. **Rule-Based Contractual Breach**: Evaluates at the instant of job reconciliation. If Actual Loss > Allowed Loss, an **EXCESS_LOSS** alert is raised immediately, tagging the specific Job Code, Karigar Code, and exact gram variance.
2. **Statistical Karigar Benchmark Drift**: Computes a rolling 30-day mean wastage percentage for each artisan across their last 5 or more completed jobs. If a karigar's cumulative loss average drifts higher than 1.5 standard deviations above their historical baseline, a **BENCHMARK_DRIFT** warning is dispatched to the Finance Head.

## 6. AI Scale-Slip Extraction & Human-in-the-Loop Workflow

The digital scale extraction microservice runs independently via FastAPI. It accepts workshop photographs of digital weight indicator displays and printed weight slips.
• **Non-Authoritative Draft Generation**: The AI service never writes directly to the database. It produces a structured JSON draft response.
• **Confidence Scoring & Gap Detection**: The microservice analyzes text clarity and field completeness. If purity or tare weight is ambiguous, missing_fields: ['purity'] is populated, and requires_human_confirmation: true is asserted.
• **UI Pre-Fill & Mandatory Clerk Review**: The web UI displays the uploaded image side-by-side with extracted draft inputs. The clerk must inspect, edit if necessary, and explicitly confirm before submission.
• **Transparent Resilience**: If the AI microservice is unreachable, the Spring Boot backend transparently falls back to local Java regex parsing without crashing the user session.

## 7. Implementation of 7 Required Application Modules

| Module Name | Route | Key Capabilities & Verified Business Logic |
|---|---|---|
| **1. Dashboard** | /dashboard | Live enterprise KPI cards (Total Stock, Issued Gold, Recovered, Outstanding, Wastage, Active Alerts). Recent transaction table. 100% backend-driven. |
| **2. Karigar Master** | /karigars | Artisan directory with KRG-xxx code generation, phone, specialty, active job count, status badges, and search filtering. |
| **3. Issue Gold** | /issue | Issuance modal with Karigar selection, gross weight, 22K/18K/24K purity presets, agreed wastage %, calculated fine gold preview, and AI slip scanner. |
| **4. Receive Gold** | /receive | Multi-stream receipt (Jewellery, Scrap, Filings), live outstanding balance counter, scale slip OCR scanner, and one-click reconciliation modal. |
| **5. Karigar Ledger** | /ledger | Running debit/credit ledger per artisan, balance summary cards, expandable job details, and downloadable transaction receipts. |
| **6. Loss Alerts** | /alerts | Centralized anomaly cockpit displaying rule-based excess loss breaches and statistical drift flags, with finance head acknowledgment. |
| **7. Stock Statement** | /statement | 5-pillar daily reconciliation statement, component recovery breakdown, CSV download, and native Tally ERP 9 / Tally Prime XML export. |

## 8. Security, Role-Based Access Control & Isolation

Authentication is enforced through stateless JWT tokens signed with HMAC-SHA256. The system defines three strictly separated roles:
• **ROLE_FINANCE_HEAD**: Full company-wide visibility. Access to company stock statements, loss anomaly dashboards, Tally XML exports, and karigar drift analytics.
• **ROLE_ISSUE_CLERK**: Operational workshop access. Can register karigars, issue raw gold, log returns, run AI slip extraction, and execute reconciliations.
• **ROLE_KARIGAR**: Strict artisan isolation. Authenticated karigars can only view their own outstanding balance and assigned jobs. Cross-karigar data access is blocked server-side by checking the JWT's karigarId claim.

## 9. Automated Test Suites & Quality Assurance Evidence

The system has been subjected to comprehensive automated test suites across all tiers:

| Subsystem | Test Class / Suite | Tests | Failures | Errors | Status |
|---|---|---|---|---|---|
| **Backend Accounting** | GoldAccountingCalculatorTest | 12 | 0 | 0 | **PASS** |
| **Backend Security** | AuthenticationIntegrationTest | 5 | 0 | 0 | **PASS** |
| **Backend Workflow** | GoldDomainWorkflowTest | 3 | 0 | 0 | **PASS** |
| **Backend Isolation** | KarigarSecurityIsolationTest | 4 | 0 | 0 | **PASS** |
| **Backend Lifecycle** | JobLifecycleAndConcurrencyTest | 3 | 0 | 0 | **PASS** |
| **AI Microservice** | pytest (test_api.py & test_extractor.py) | 8 | 0 | 0 | **PASS** |
| **Frontend UI** | TypeScript compiler & Vite build | 1512 mod | 0 | 0 | **PASS** |

## 10. Known Limitations, Operational Assumptions & Roadmap

**Assumptions & Current Scope:**
• *Single Central Workshop*: Currently models 1 central manufacturing facility feeding 5 retail showrooms.
• *Homogeneous Purity per Job*: Each job is issued at a single uniform karatage (e.g. 22K 91.60%). Multi-alloy composite jobs are modeled as linked sub-jobs.

**Future Roadmap:**
• *Direct RS-232 / USB Serial Scale Integration*: Stream live weight readings directly from Mettler Toledo and Sartorius scales into the browser.
• *Native Mobile App for Artisans*: Offline-first React Native mobile client enabling karigars in remote workshop clusters to acknowledge receipt of gold via QR code.

## 11. Submission Verification & Repository Details

• **Public GitHub Repository**: https://github.com/Ayush5424/jewellery-gold-accountability
• **Branch**: main
• **Primary Developer**: Ayush
• **Assignment**: Isocode Labs Software Engineering Internship Assignment
• **Date of Delivery**: November 2026